

Checklist de madurez MLOps

Para auditar un proyecto de ML real — el tuyo o el de tu equipo · Taller MLOps UNI

Cómo usarlo: marca cada práctica solo si tienes la evidencia a la derecha. Los niveles son acumulativos: no persigas el nivel 3 con huecos en el 1. Úsalo el lunes en tu proyecto actual: 30 minutos de auditoría honesta valen más que cualquier herramienta nueva.

Nivel 1 · Reproducible (la base sin la cual nada importa)

✓	Práctica	Evidencia de que se cumple
☐	El código está en Git y el notebook no es el producto final	Existe un repo con src/ modular; el notebook es exploración, no producción
☐	La configuración vive en un archivo versionado	params.yaml (o similar) en Git; cambiar un hiperparámetro no toca código
☐	Semillas fijas en split y modelos	Dos corridas seguidas dan métricas idénticas
☐	Dependencias pinneadas	requirements.txt con versiones exactas; entorno recreable en otra máquina
☐	Los datos se validan a la entrada	Una función/herramienta que falla ruidosamente ante columnas faltantes, tipos malos o rangos absurdos
☐	El preprocesamiento vive DENTRO del pipeline del modelo	El artefacto guardado acepta datos crudos (con nulos y categorías nuevas)

Nivel 2 · Gestionado (experimentos y modelos con historia)

✓	Práctica	Evidencia de que se cumple
☐	Cada entrenamiento queda registrado automáticamente	Tracking (MLflow o equivalente) con parámetros, métricas y artefactos por run
☐	La selección de modelo sigue un criterio escrito	El criterio está en el README/wiki y se aplica por código, no por corazonada
☐	Los modelos tienen nombre, versión y estado	Registry con alias (champion/challenger) o convención equivalente
☐	El umbral de decisión se acordó con negocio	Documento o ticket donde consta el costo de FP vs FN y el umbral elegido
☐	Hay tests del contrato de datos, del modelo y del servicio	pytest corre en local y los tests fallan si el contrato se rompe

Nivel 3 · Automatizado (nadie despliega a mano)

✓	Práctica	Evidencia de que se cumple
☐	CI corre en cada push	Lint + tests verdes obligatorios para mergear
☐	El CI incluye un quality gate del modelo	Un entrenamiento smoke falla el pipeline si la métrica cae bajo el mínimo
☐	El servicio se construye como imagen	Dockerfile versionado; la imagen la construye el CI, no una laptop
☐	Desplegar y revertir no requieren tocar código	Mover un alias / re-apuntar una versión basta; hay historial de quién y cuándo

✓	Práctica	Evidencia de que se cumple
☐	El modelo en producción es identificable	Cada predicción o lote registra qué versión de modelo la produjo

Nivel 4 · Monitoreado (el ciclo se cierra)

✓	Práctica	Evidencia de que se cumple
☐	Se monitorean los DATOS, no solo la infraestructura	Reporte de drift periódico (Evidently o equivalente) contra la referencia de entrenamiento
☐	Se monitorea la distribución de las predicciones	Alerta si la media/percentiles se mueven más del umbral acordado
☐	Existe una política escrita señal → acción	Tabla con umbrales y responsables; no depende de que alguien 'mire el dashboard'
☐	El desempeño real se mide cuando llega la verdad del terreno	Job que recalcula métricas con ground truth y las compara contra el baseline
☐	Re-entrenar es rutina, no heroísmo	El re-entrenamiento reutiliza el pipeline versionado y compite contra el champion antes de promoverse

Lectura del resultado

Tu foto	Diagnóstico y siguiente paso
Nivel 1 incompleto	Riesgo alto: resultados no defendibles. Prioridad absoluta: pipeline + seeds + validación (Lab 1). Es una semana de trabajo, no un proyecto.
Nivel 1 ✓, nivel 2 a medias	El caso más común. Siguiendo paso de mayor retorno: tracking de experimentos (Lab 2) — se instala en una tarde.
Niveles 1-2 ✓	Listo para automatizar: monta la plantilla del taller con CI (Lab 4) en tu próximo proyecto.
Niveles 1-3 ✓	Estás sobre la media de la industria. Cierra el ciclo con monitoreo y política de re-entrenamiento (Lab 5).

Material del taller MLOps en la práctica · UNI · libre para uso y adaptación en tu organización.